

SENIOR SOFTWARE DEVELOPER TAKE-HOME TECHNICAL ASSESSMENT

4-Day Engineering Challenge

Technology focus: Angular / Next.js / Java Spring Boot / Node.js

Assessment areas: backend engineering, frontend delivery, architecture, testing, DevOps, security, documentation, and practical AI integration.

Candidate instruction

You will be assigned ONE project from the five options in this document. The goal is not to build a huge product. The goal is to demonstrate how you design, implement, test, explain, and ship a production-minded solution under a four-day constraint.

Confidential recruitment material

1. Purpose of the Assessment

This assessment is intended to evaluate senior-level engineering judgement, not just feature completion. We want to understand how you break down ambiguous requirements, make trade-offs, model data, structure code, handle failure, build usable interfaces, test critical paths, and communicate technical decisions.

The project deliberately sits outside our day-to-day business domains. You are therefore assessed on transferable engineering ability rather than prior knowledge of our internal products.

2. Timebox and Scope

- Duration: 4 calendar days from the time the assignment is issued.
- Build only the features required for a convincing working vertical slice. Do not spend the entire time polishing low-value extras.
- A partially complete solution with strong architecture, tests, documentation, and clearly stated trade-offs can score better than a large but fragile implementation.
- You may use open-source libraries where appropriate. Be ready to explain why you selected them.
- Docker is strongly encouraged so reviewers can run the solution consistently.

3. Technology Expectations

Use technologies from our working stack so we can evaluate skills that transfer directly into the role.

Area	Requirement
Backend	Java 21+ with Spring Boot OR Node.js with TypeScript. If Node.js is selected, use a structured framework such as NestJS, Fastify, or Express with a clear architecture.
Frontend	Angular OR Next.js with TypeScript. The UI must be functional and integrated with the backend; a static mock-up is not sufficient.
Database	PostgreSQL is preferred. Another relational database is acceptable if justified. Use migrations.
Caching / Queue	Redis, a message broker, or an in-process equivalent may be used where the project benefits from it. Explain the production alternative if you simplify locally.
AI	Include the project-specific AI feature described below. Any reputable LLM provider or a local model may be used. The application must fail gracefully when the AI service is unavailable.
DevOps	Docker / Docker Compose preferred. Include environment configuration and one-command or clearly documented startup instructions.

4. General Requirements for Every Project

- Authentication: implement a lightweight but real authentication mechanism for the admin/user-facing application. JWT/session authentication is acceptable.
- Authorization: protect at least one operation by role or permission, e.g. ADMIN versus VIEWER.
- Validation: validate request payloads on the backend and show useful validation feedback in the frontend.
- Error handling: expose consistent API errors; do not return raw stack traces to the client.
- Persistence: data must survive application restarts unless a requirement explicitly describes ephemeral data.
- API design: use sensible resource naming, HTTP methods/status codes, pagination where data can grow, and clear request/response models.
- Observability: include structured logs and enough information to diagnose failed operations. Bonus for correlation/request IDs and basic metrics.
- Security: do not commit secrets. Consider common API and web risks such as injection, broken access control, insecure direct object references, unsafe file input, SSRF where URLs are accepted, and prompt injection around AI features.
- Testing: include automated tests for important business logic plus at least a small number of API/integration tests. Frontend tests are encouraged for non-trivial states or components.
- Usability: the UI should be coherent and usable on a normal laptop screen. Visual perfection is not required, but loading, empty, error, and success states should be handled.

Project 1: Feature Flag & Configuration Platform

Objective: Build a small platform that allows teams to control application features dynamically without redeploying software.

Scenario: An engineering team wants to create feature flags, target selected users, gradually roll out features, and inspect who changed configuration over time.

Minimum functional scope

Area	Requirement
Backend	Create APIs to manage flags and environments (for example DEV and PROD), evaluate flags for a supplied user context, and record an immutable audit trail. Support at least BOOLEAN and PERCENTAGE_ROLLOUT flag strategies. Include caching and demonstrate a clear cache invalidation approach.
Frontend	Build an admin dashboard showing flags, status, environment, rollout strategy, last modified information, create/edit screens, and an audit-log view. Include a simple evaluation playground where a reviewer can enter user attributes and see the resolved flag result.
AI capability	Add an AI "rule assistant" that converts a natural-language rollout request such as "enable this for 20% of users in Harare except internal staff" into a proposed structured rule. The AI must only PROPOSE configuration; a human must review and save it. Show the raw structured proposal and validate it before persistence.

Required implementation details

- Evaluation endpoint should be designed for low latency; explain how you would keep it performant at scale.
- Avoid percentage assignment that changes randomly between requests. A user should remain consistently inside/outside the rollout bucket.
- Every configuration change must create an audit record containing actor, timestamp, previous value, and new value.
- Demonstrate at least one concurrency or stale-update protection mechanism (version column, ETag, optimistic locking, etc.).

Optional stretch goals

- Server-Sent Events or WebSocket updates when a flag changes.
- SDK-style sample client showing how an application would consume the evaluation API.
- Basic evaluation metrics such as count per flag/result.

Project 2: Multi-Tenant Notification & Webhook Dispatcher

Objective: Build a reliable asynchronous delivery service that accepts events and distributes notifications/webhooks for multiple tenants.

Scenario: Different organizations submit events to the platform. Each organization configures destinations and needs visibility into delivery status and failures.

Minimum functional scope

Area	Requirement
Backend	Create tenant-aware APIs for destinations, event submission, delivery attempts, retrying failures, and delivery logs. Process dispatches asynchronously. Implement retry with exponential backoff and a terminal/dead-letter state. Protect tenant data boundaries.
Frontend	Build a dashboard for tenant selection, configured destinations, recent events, delivery attempts, status filters, attempt details, and a manual retry action. Provide a compact operational summary such as success/failure counts.
AI capability	Add an AI "failure summarizer" that reads sanitized delivery metadata/error messages for a failed event and produces a short explanation plus suggested operator action. Do not send secrets, authorization headers, or full sensitive payloads to the model. Clearly identify AI-generated content in the UI.

Required implementation details

- Prevent one tenant from accessing another tenant's endpoints or logs.
- Capture every delivery attempt, response status, duration, and next retry time.
- Implement idempotency for submitted events or clearly show how duplicate dispatches are prevented.
- If arbitrary webhook URLs are accepted, document and implement reasonable SSRF protections.
- Demonstrate rate limiting or per-destination throttling.

Optional stretch goals

- HMAC signing of outgoing webhook requests.
- Replay of a historical event to a selected destination.
- Simple circuit-breaker behavior for repeatedly failing destinations.

Project 3: Distributed Job Scheduler & Operations Console

Objective: Build a scheduler for one-off and recurring background jobs with safe execution tracking.

Scenario: Operations staff need to define scheduled jobs, trigger them manually, see what is running, and investigate failures without direct server access.

Minimum functional scope

Area	Requirement
Backend	Create APIs to define jobs, schedule one-off or recurring execution, run immediately, cancel when appropriate, and inspect execution history. Track PENDING, RUNNING, SUCCESS, FAILED, and optionally CANCELLED states. Prevent the same scheduled execution from being processed twice.
Frontend	Build a control panel with jobs list, next run time, enable/disable control, run-now action, execution history, duration, filters, and status detail. Live/polling refresh should make running jobs visibly progress.
AI capability	Add an AI "schedule assistant" that turns a natural-language request such as "run the daily reconciliation report every weekday at 6:30 PM" into a proposed schedule/cron expression and plain-English interpretation. The generated schedule must be validated and explicitly confirmed before activation.

Required implementation details

- Implement at least two safe demo job types, for example DELAYED_TASK and DATA_SUMMARY. Do not execute arbitrary shell commands supplied by users.
- Use database locking, leader election, or another defensible approach to avoid duplicate execution in a multi-instance design.
- Store start time, end time, duration, status, and failure reason for every execution.
- Document your approach to clock/timezone handling.

Optional stretch goals

- Retry policy per job.
- Concurrency limits so one job type cannot exhaust all workers.
- Basic metrics dashboard such as average duration and failure rate.

Project 4: Structured Document Versioning & Review Workspace

Objective: Build a system that stores structured JSON documents with immutable versions and useful change review.

Scenario: A team edits structured records over time and needs a trustworthy history showing exactly what changed, by whom, and when.

Minimum functional scope

Area	Requirement
Backend	Create APIs to create documents, update them, list versions, retrieve a historical version, compare two versions, and optionally restore an old version by creating a new current version. Versions must be immutable. Use optimistic concurrency to avoid silently overwriting another user's edits.
Frontend	Build an editor for a reasonably complex structured document, a version timeline, and a side-by-side or inline diff view that highlights added, removed, and changed fields. Include clear conflict feedback if a stale version is submitted.
AI capability	Add an AI "change summary" that takes a computed diff (not the whole database record) and writes a concise human-readable summary of meaningful changes. The application should make clear that the diff itself is authoritative and the AI summary is explanatory only.

Required implementation details

- Use server-side version numbers/revisions and never mutate historical rows.
- Persist actor and timestamp for each version.
- Diff nested objects and arrays in a way that is understandable to reviewers.
- Handle concurrent edits explicitly rather than last-write-wins without warning.

Optional stretch goals

- Comments or approval state attached to a specific version.
- JSON schema-based validation for document structure.
- Search across current document metadata.

Project 5: Media Ingestion & Processing Workspace

Objective: Build a small media asset service that safely ingests files, extracts metadata, and supports operational review.

Scenario: A content team uploads image/document assets and needs progress visibility, metadata, duplicate detection, and a searchable library.

Minimum functional scope

Area	Requirement
Backend	Create APIs for upload, metadata retrieval, search/filter, deletion, and processing status. Extract safe metadata such as MIME type, size, checksum, and image dimensions where applicable. Store files outside the database or using an object-storage abstraction. Include duplicate detection using checksum.
Frontend	Build an asset library with drag-and-drop upload, progress indication, processing state, thumbnail/preview where appropriate, metadata panel, filters, error state, and delete action. Large collections should be paginated or virtualized sensibly.
AI capability	Add an AI "metadata assistant" for image assets: generate a proposed short title, description, and tags from an image or provided extracted text. AI output must remain editable before save. If vision access is not available, provide a clearly documented mock/provider abstraction and demonstrate the integration flow.

Required implementation details

- Validate file type and file size on the server; do not trust the browser MIME type alone.
- Generate and persist a checksum; use it to identify duplicate uploads.
- Do not expose arbitrary server file paths to the client.
- Processing failures must have a visible status and useful diagnostic message.
- Explain how the design would evolve to object storage and asynchronous workers at larger scale.

Optional stretch goals

- Chunked/resumable uploads.
- Background thumbnail generation.
- Pre-signed object-storage URLs or equivalent secure download design.

5. AI Engineering Expectations

The AI component is intentionally small. We are not evaluating who can place the biggest LLM call. We are evaluating whether you can integrate probabilistic AI behavior safely and cleanly into normal software.

- Wrap model/provider access behind a service interface rather than calling an LLM directly throughout controllers/components.
- Use structured output where the feature produces data that will be persisted or executed. Validate the output against normal application rules.
- Do not let model output directly perform privileged actions without deterministic validation and user confirmation.
- Handle timeouts, malformed responses, rate limits, and provider unavailability.
- Protect secrets and avoid sending unnecessary sensitive data to an external model.
- Document the prompt strategy, model/provider used, expected output shape, failure behavior, and any cost/latency trade-offs.
- Include at least one test where the AI provider is mocked/stubbed.

6. Repository & Submission Requirements

Submit a Git repository accessible to the review team. A mono-repository is recommended, but separate frontend/backend repositories are acceptable if clearly linked.

- Repository must contain all source code, migrations, tests, Docker/configuration files, and documentation required to evaluate the submission.
- Do not commit .env files, API keys, passwords, generated dependency folders, IDE metadata, or large binaries that can be generated locally.
- Include a .env.example with every required environment variable and safe placeholder values.
- Use meaningful commits. We want to see how the solution evolved; a single final mega-commit makes that harder.
- Create a final Git tag or release named submission (or clearly identify the final commit SHA in the README).
- If a hosted demo is provided, still ensure the project can be run locally from the repository.

7. Required README.md

Your README is part of the assessment. A reviewer should be able to understand and run your project without messaging you for missing steps.

- Project overview and the problem being solved.
- Chosen stack and brief justification for major technology decisions.
- Architecture overview, including a simple diagram (Mermaid is acceptable).
- Prerequisites and exact local setup instructions.
- Environment variables and how to obtain/configure optional AI credentials.
- Database migration and seed/demo-data instructions.
- How to start backend, frontend, worker/queue dependencies, and any supporting services.
- Demo users/credentials if authentication is implemented locally.
- API documentation: OpenAPI/Swagger preferred, plus any non-obvious endpoints or flows.
- How to run all automated tests.
- AI integration: provider/model, prompt/output design, safety/validation, fallback behavior, and how to run without a paid API key if possible.
- Important trade-offs, known limitations, and what you would improve with another week.
- A short "production readiness" section covering scaling, observability, security, data backup/recovery, and deployment considerations.

8. Mandatory Deliverables

- Working backend service.
- Working frontend application integrated with the backend.
- Database schema and migrations.
- Project-specific AI feature.
- Automated tests.
- README.md meeting the requirements above.
- API documentation (Swagger/OpenAPI or equivalent).
- Docker-based run configuration where practical.
- A short 5-10 minute screen recording OR live demo during review showing the main flow, one failure/error path, the AI feature, and the tests running.

9. What We Will Review in the Demo

- Can the candidate explain the architecture without hiding behind framework terminology?
- Can they trace one request end-to-end through UI, API, persistence, and background/AI components?
- Can they explain one important design trade-off and an alternative they rejected?
- Can they show what happens when a dependency fails?
- Can they explain how they would scale the system and what would break first?
- Can they identify security risks in their own solution?
- Can they modify a small piece of behavior during the review without destabilizing the project?

10. Scoring Rubric (100 Points)

Category	Weight	What Strong Work Looks Like
Backend engineering & domain design	20	Clear boundaries, maintainable structure, appropriate data model, correct concurrency/failure handling, sensible API design.
Frontend engineering & UX	15	Well-structured TypeScript, component/state discipline, integrated workflows, loading/error/empty states, usable responsive UI.
System design & senior-level judgement	15	Good trade-offs, appropriate complexity, scalability thinking, consistency and failure-mode awareness.
Testing & correctness	15	Meaningful unit/integration coverage focused on risk, reproducible test setup, edge cases handled.
AI integration	10	Provider abstraction, structured/validated outputs,

		human control where needed, safe failure behavior, testability.
Security & resilience	10	Authorization boundaries, input validation, secrets hygiene, relevant threat awareness, defensive design.
DevOps & operability	5	Simple run/deploy path, containers/configuration, logs/health, reproducible environment.
Documentation & communication	10	Excellent README, clear setup, architecture explanation, trade-offs, limitations, professional demo.

Scoring note: A solution does not need every optional feature to score highly. Strong fundamentals, clear reasoning, reliable behavior, and good communication matter more than feature volume.

11. Automatic / Major Deductions

- Application cannot be run from the supplied instructions without substantial reverse engineering.
- Secrets or private credentials are committed to the repository.
- Frontend is only a static mock and does not integrate with the submitted backend.
- Core data is hard-coded instead of persisted without explanation.
- No meaningful automated tests.
- AI output is trusted as authoritative without validation in a feature that changes configuration or triggers behavior.
- Candidates cannot explain significant portions of the submitted code or architecture.
- Copied/generated code is submitted without understanding or with incompatible/unused sections left in the project.

12. Use of AI Coding Tools

AI-assisted development tools are allowed. Senior engineers are expected to use modern tools effectively, but they remain responsible for every line they submit.

- In the README, briefly disclose the tools used (for example ChatGPT, Claude, Copilot, Cursor, Codex, Gemini, etc.) and what you used them for.
- You must be able to explain, debug, and modify all submitted code during review.
- Do not include confidential information from current or previous employers in prompts, code, sample data, or documentation.
- Evaluation focuses on your engineering judgement and ownership, not whether code was typed manually.

13. Recommended Repository Structure

```
root/  
  README.md  
  .env.example  
  docker-compose.yml  
  docs/  
    architecture.md (optional)  
  backend/  
    src/  
    tests/  
    Dockerfile  
  frontend/  
    src/  
    tests/  
    Dockerfile  
  scripts/ (optional)
```

14. Final Submission Checklist

- ☐ Repository access has been granted to the review team.
- ☐ Final commit/tag is clearly identified.
- ☐ README setup steps work from a clean environment.
- ☐ .env.example contains all required keys and no real secrets.
- ☐ Migrations and seed/demo data work.
- ☐ Backend and frontend start successfully.
- ☐ AI feature can be demonstrated or has a documented safe fallback/mock mode.
- ☐ Automated tests run using the documented command.
- ☐ Demo credentials are provided where needed.
- ☐ Known limitations and trade-offs are documented.